

UNIVERSIDAD DON BOSCO

Facultad de Ingeniería

Escuela de Computación

Lenguajes Interpretados en el Cliente

LIC104 G01T

Presentado Por:

Flores Turcios, Herberth Rodrigo (FT240273)

Suria Mendoza, Daniel Alejandro (SM240271)

Sosa Alvarenga, Emerson Alejandro (SA240274)

Velasco García, Francisco Josué (VG240485)

Docente:

Ing. Carmen Celia Morales Samayoa

San Salvador, 19 de octubre del 2026

INDICE

1. DESCRIPCIÓN DEL EJERCICIO 1

1.1. Planteamiento y alcance del ejercicio 1

1.2. Funcionalidades del sistema 1

2. PROPUESTA DE MAQUETADO / WIREFRAMES 2

2.1. Enlace al prototipo interactivo en Figma 2

2.2. Propuesta visual e identidad gráfica 2

2.3. Codificación cromática territorial y continuidad gráfica 3

2.4. Distribución de multidispositivo (Wireframes) 3

2.5. Flujo de trabajo de usuario 6

2.6. Funcionalidades del sistema 9

3. JUSTIFICACIÓN DE ARQUITECTURA Y SEMÁNTICA HTML5 11

3.1. Encabezado y barra de navegación 11

3.2. Área de contenido principal 11

3.3. Bloques temáticos con id's 11

3.4. Mapa SVG 12

3.5. Nomenclatura de clases e identificadores 12

3.6. Carga de fuentes tipográficas 13

3.7. Pie de página 13

4. FUNDAMENTACIÓN DEL DISEÑO Y ESTILOS CSS 13

4.1. Ajuste responsivo del SVG del mapa 13

4.2. Línea divisoria con texto centrado 13

4.3. Jerarquía tipográfica (encabezados y párrafos) 14

4.4. Barra de navegación fija y personalizada 15

4.5. Identificadores de departamento, submenús y contenedores dinámicos 16

4.6. Submenú navegable de categorías. 17

4.7. Cajas contenedoras estáticas y placeholders dinámicos. 17

4.8. Colorización por zona geográfica departamental 18

4.9. Personalización de Popovers informativos de Bootstrap. 18
4.10. Desplazamiento suave y desfases de secciones 19
4.11. Estructuración del contenedor de contenido dinámico 20
4.12. Formateo y adaptabilidad visual de la silueta e imagenes 20
4.13. Optimización para interacciones táctiles en dispositivos moviles 21
4.14. Enmarcado responsivo para videos y créditos 22
4.15. Efecto de elevación 3D en los departamentos del mapa 22
5. Funcionamiento del sitio web y estructura de los archivos js. 23
5.1. menuPrincipal.js (Controlador del Mapa Interactivo y Navegación) 23
5.1.1. Fragmentos claves: 23
5.1.2. Lógica adaptable para pantallas táctiles y mouse. 23
5.2. Api.js (Capa de Servicios y Acceso a Datos) 25
5.2.1. Fragmentos Claves: 25
5.2.2. Petición AJAX asíncrona desacoplada con Promesas: 25
5.2.3. Caché en memoria y fallback a JSON local: 26
5.3. detalleDepto.js (Controlador de la Vista Departamental) 27
5.3.1. Fragmentos claves: 27
5.3.2. Lectura de parámetros URL y orquestación inicial: 27
5.3.3. Renderizado vectorial y autoencuadre geométrico (viewBox): 28
5.3.4. Inyección dinámica del contenido y eliminación de los placeholders de carga: 29
5.4. siluetas.js (Repositorio de Geometrías Vectoriales) 31
6. BIBLIOGRAFÍA 32
7. ANEXOS 34

1. DESCRIPCIÓN DEL EJERCICIO

1.1. Planteamiento y alcance del ejercicio

El presente proyecto consiste en el desarrollo de un sitio web interactivo enfocado a la descripción geográfica, demográfica, cultural e histórica de la Republica de El Salvador. El sitio busca solucionar la necesidad de centralizar la información departamental del país en una interfaz visual limpia moderna, intuitiva y dinámica, permitiendo a los usuarios explorar el territorio nacional.

El sitio abarcará los 14 departamentos de El Salvador distribuidos en sus tres zonas geográficas (Occidental, Central y Oriental), ofreciendo dos niveles de consulta: una vista general panorámica con datos estadísticos rápidos y vistas específicas por departamento con información cultural e histórica profunda.

1.2. Funcionalidades del sistema

Pantalla principal con mapa vectorial interactivo:

- Despliegue del mapa general de El Salvador renderizado mediante vectores SVG.
- Detección del cursor sobre cada departamento (*hover*), activando transiciones visuales de color y efectos de relieve/sombra.
- Despliegue de tarjetas flotantes (*tooltips*) con información rápida:
    - ✓ Nombre del departamento y cabecera departamental.
    - ✓ Extensión territorial (km^2).
    - ✓ Población estimada / número de habitantes.
    - ✓ Fecha de fiestas patronales.

Sistema de navegación global y acceso rápido por zonas:

- Barra de navegación superior con menú desplegable clasificado por zonas geográficas:
    - ✓ **Zona Occidental:** Ahuachapán, Santa Ana, Sonsonate.
    - ✓ **Zona Central:** La Libertad, Chalatenango, Cuscatlán, San Salvador, La Paz, Cabañas, San Vicente.
    - ✓ **Zona Oriental:** Usulután, San Miguel, Morazán, La Unión.
- Permite al usuario acceder directamente a cualquier departamento sin interactuar obligatoriamente con el mapa visual.

Vistas individuales y detalladas por departamento:

Al hacer clic en un departamento (desde el mapa o desde el menú), el sistema carga una página individual con submenús internos organizados en las siguientes categorías:

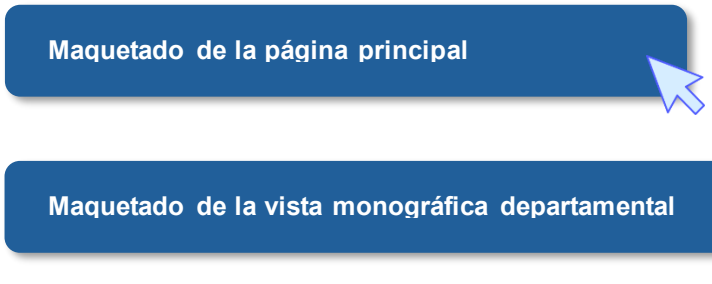
- **Mapa local:** Silueta y división municipal del departamento seleccionado.
- **Reseña histórica:** Fechas de fundación, antecedentes coloniales y sucesos relevantes.
- **Municipios:** Listado completo de los municipios que lo conforman.
- **Turismo y naturaleza:** Principales centros turísticos, lagos, ríos, volcanes y cerros representativos.
- **Personajes célebres:** Biografías resumidas de escritores, poetas, artistas, músicos, cantantes y figuras destacadas originarias de la región.

Módulos complementarios:

- **Página de Desarrolladores:** Fichas de presentación del equipo de trabajo (nombres, roles, fotografía y contacto).
- **Página de Contáctanos:** Formulario web con validaciones de campos (nombre, correo electrónico, asunto y mensaje) para retroalimentación de los usuarios.

2. PROPUESTA DE MAQUETADO / WIREFRAMES

2.1. Enlace al prototipo interactivo en Figma



2.2. Propuesta visual e identidad gráfica

Cabecera temático-cultural (*Header Banner*): Incorpora una franja ilustrada con iconografía y texturas precolombinas, delimitando la marca institucional ("NUESTRAS RAÍCES") a la izquierda y el enlace directo de navegación ("INICIO") a la derecha, manteniéndose persistente tanto en la pantalla de inicio como en las vistas departamentales.

2.3. Codificación cromática territorial y continuidad gráfica

El territorio nacional se organiza en tres macrorregiones visuales que se conservan tanto en el mapa general como en las siluetas vectoriales individuales de cada monografía:

- **Zona Occidental (Verde #6dc998):** Ahuachapán, Santa Ana y Sonsonate.
- **Zona Central y Paracentral (Naranja / Terracota #d97736):** Chalatenango, La Libertad, San Salvador, Cuscatlán, La Paz, Cabañas y San Vicente.
- **Zona Oriental (Turquesa / Azul Verdoso #1f8a85):** Usulután, San Miguel, Morazán y La Unión.

Estructuración modular de contenido: La vista monográfica implementa contenedores uniformes en bloque (*Texto e imágenes*) que alojan información monográfica enriquecida, garantizando una lectura ordenada y modular.

2.4. Distribución de multidispositivo (Wireframes)

Vista Escritorio (*Desktop* mayor o igual a 1440px)

Pantalla Principal: Distribución simétrica en dos columnas paralelas (col-lg-6) que ubica el bloque textual a la izquierda y el mapa interactivo a la derecha, permitiendo una exploración panorámica continua sin requerir *scroll*.

Vista Monográfica: Presentación superior en dos columnas donde la silueta vectorial individual del departamento se ubica a la izquierda y la descripción contextual general a la derecha. En la parte inferior, el menú de anclaje rápido (Conocé más aquí:) se despliega de manera horizontal en una sola línea (•Historia •Tradiciones y cultura •Gastronomía •Lugares de Interés), dando paso a los módulos de contenido apilados secuencialmente a ancho completo.





Vista Tableta (*Tablet* entre 769px a 1439px):

- **Pantalla Principal:** Esquema de una sola columna apilada y centrada para evitar que el mapa reduzca excesivamente sus trazados vectoriales en pantallas intermedias, situando títulos superiores, mapa central y narrativa en la base.

- **Vista Monográfica:** Reorganización en un flujo vertical centrado. La silueta del departamento se posiciona en el eje central superior debajo de su identificador, seguida del párrafo introductorio. El menú de navegación interna (Conocé más aquí:) se adapta a una lista vertical apilada para facilitar la selección táctil, manteniendo los bloques de contenido (Historia, Tradiciones y cultura, etc.) en tarjetas completas que aprovechan el ancho disponible.

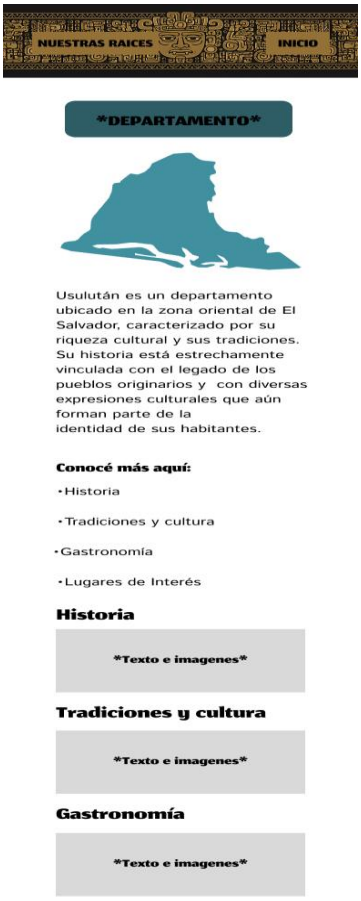




Vista Móvil (*Mobile* entre 1px a 767px)

- **Pantalla Principal**: Diseño vertical compacto bajo el principio *Mobile-First*. El mapa SVG escala dinámicamente al 100% del ancho del área visible, optimizando la interacción táctil con una sola mano.
- **Vista Monográfica:** Disposición en columna única vertical estricta. El identificador (*DEPARTAMENTO*) y el gráfico del territorio reducen proporcionalmente sus dimensiones para evitar desbordamientos visuales. El menú de anclajes secundarios se organiza en enlaces táctiles independientes dispuestos en lista vertical, facilitando el salto directo hacia las secciones de texto e imágenes sin saturar la pantalla pequeña





2.5. Flujo de trabajo de usuario

El flujo de interacción del aplicativo web guía al usuario a través de una experiencia progresiva de tres etapas, transitando desde la exploración geoespacial panorámica hasta la consulta monográfica detallada de cada departamento:

Exploración panorámica y zonificación regional (Vista Inicial)

- **Visualización del mapa nacional:** Al ingresar al sistema, el usuario se encuentra con el mapa completo de El Salvador segmentado cromáticamente en tres regiones (Occidental en verde, Central en terracota y Oriental en turquesa) para una inmediata identificación territorial.
- **Reconocimiento interactivo:** Al mover el cursor (o pulsar en pantallas táctiles) sobre la superficie de cualquier departamento, el sistema reconoce el trazado vectorial activo.



Despliegue contextual y vista previa (*Tooltip / Hover State*)

- **Activación de la tarjeta emergente:** Al posicionar el cursor o interactuar sobre un territorio específico (por ejemplo, el departamento de Usulután), la silueta vectorial del departamento experimenta una transición visual que le confiere un efecto 3D inmediato, elevándose sutilmente sobre el plano mediante escalado, desplazamiento vertical y sombras proyectadas multinivel (drop-shadow) que la despegan del resto del mapa. De forma simultánea, se despliega un cuadro flotante contextual (*tooltip*) con fondo oscuro y tipografía contrastante.

- **Contenido de la previsualización:** La ventana muestra de manera sintetizada:

  - ✓ El nombre del departamento seleccionado (*Departamento*).
  - ✓ Una breve síntesis descriptiva con datos demográficos o culturales clave.



.

Carga y navegación de la Vista Monográfica (Página de Detalle)

Al hacer clic sobre el enlace Ver más... o sobre la silueta departamental, el sistema transiciona hacia la vista monográfica exclusiva de esa región, organizada bajo la siguiente jerarquía:

- **Cabecera temático-cultural:** Barra superior persistente con iconografía prehispánica, identificador de marca ("NUESTRAS RAÍCES") y botón de retorno ("INICIO") para volver a la pantalla principal en cualquier momento.

- **Módulo de identidad territorial:**

  Lateral izquierdo: Silueta vectorial aislada del departamento con el color representativo de su zona geográfica.

  Lateral derecho: Etiqueta del departamento y párrafo introductorio que resume su historia, ubicación geográfica y legado de pueblos originarios.

- **Barra de navegación interna por subtemas (*Submenu Anchor Links*):** Menú horizontal de acceso rápido rotulado como “Conocé más aquí:”, el cual permite desplazarse directamente mediante anclas hacia cuatro ejes temáticos:

  - ✓ Historia: Registro de fundación, orígenes y antecedentes cronológicos acompañados de material gráfico.
  - ✓ Tradiciones y cultura: Catálogo de expresiones folclóricas, festividades y costumbres locales.
  - ✓ Gastronomía: Muestra de platillos típicos y tradiciones culinarias representativas.
  - ✓ Lugares de Interés: Inventario de sitios turísticos, atractivos naturales y monumentos destacados.

- **Bloques modulares de contenido:** Secciones estructuradas con encabezados dedicados (Historia, Tradiciones y cultura, etc.) y contenedores diseñados para alojar texto enriquecido, galerías fotográficas y recursos multimedia.



2.6. Funcionalidades del sistema

A partir de la arquitectura técnica, la propuesta de maquetado en Figma y el flujo de trabajo definido, el sitio web "Nuestras Raíces" integra las siguientes funcionalidades interactivas y estructurales:

Renderizado y manipulación vectorial nativa (SVG en el DOM)

- **Escalabilidad fluida sin degradación:** Renderizado del mapa de El Salvador mediante nodos vectoriales independientes (<path>), conservando nitidez absoluta en cualquier factor de ampliación (*zoom*) o densidad de píxeles sin recurrir a mapas de bits pesados.
- **Manipulación directa del DOM:** Cada departamento cuenta con identificadores estandarizados (id="SVAH", id="SVSA", etc.) y atributos de datos, permitiendo asociar transiciones de color, efectos y eventos sin intermediarios ni recargas de página.

Retroalimentación visual y relieve dinámico (*Hover States*)

- **Efecto de elevación tridimensional:** Al situar el cursor sobre cualquier departamento en interfaces de escritorio, el territorio activo reacciona mediante una escala sutil y cambio de contorno, confirmando visualmente la selección en tiempo real.

Previsualización contextual inmediata (*Tooltips* flotantes)

- **Extracción dinámica de metadatos:** Lectura asíncrona de los atributos incrustados en cada trazado vectorial para desplegar una ventana flotante oscura y contrastante al interactuar con el departamento.

- **Contenido resumido y llamado a la acción:** Despliegue del nombre oficial del territorio, una síntesis cultural/demográfica y el enlace interactivo Ver más... como disparador directo hacia la página de detalle.

Adaptabilidad táctil inteligente (*Responsive & Touch Optimization*)

- **Mapeo de eventos táctiles:** En pantallas táctiles (móviles y tabletas), el sistema reemplaza los eventos de cursor por eventos de pulsación (*tap/touch*), siendo una para ver texto emergente y dos pulsaciones para acceder a la pagina con la información del departamento

- **Tarjeta contextual inferior:** La ventana emergente se posiciona como un panel en la base de la pantalla para evitar que los dedos del usuario obstruyan la visualización del mapa durante la navegación con una sola mano.

Módulo monográfico departamental dedicado

Al confirmar la selección de una región, el sistema carga una vista monográfica exclusiva con la siguiente estructura:

- **Identidad territorial individual:** Encabezado con distintivo superior en bloque (*DEPARTAMENTO*), silueta vectorial aislada con el color de su respectiva zona y párrafo de contextualización histórica/geográfica.

- **Navegación interna por anclas temáticas (*Submenu Anchor Links*):** Barra horizontal (o lista apilada en móviles) rotulada como “Conocé más aquí:”, que permite saltar de inmediato entre cuatro ejes de contenido:

    - ✓ **Historia:** Orígenes precolombinos, fecha de fundación y antecedentes cronológicos.
    - ✓ **Tradiciones y cultura:** Manifestaciones folclóricas, identidad de pueblos originarios y fiestas patronales.
    - ✓ **Gastronomía:** Platillos típicos y herencia culinaria local.
    - ✓ **Lugares de Interés:** Lista de atractivos turísticos, sitios arqueológicos y accidentes geográficos representativos.

- ✓ **Contenedores de contenido enriquecido:** Módulos estandarizados (*Texto e imágenes*) diseñados para organizar la narrativa, galerías fotográficas y recursos multimedia.

Sistema de navegación global persistente

- **Cabecera temática institucional (*Header Banner*):** Franja superior con diseño precolombino persistente en todas las vistas, que integra el logotipo/marca ("NUESTRAS RAÍCES") y el botón de retorno rápido ("INICIO") para volver a la pantalla principal en cualquier momento del recorrido.

3. JUSTIFICACIÓN DE ARQUITECTURA Y SEMÁNTICA HTML5

La arquitectura implementada en el sitio web fundamenta su estructura en los estándares modernos de HTML5, la separación estricta de responsabilidades (estructura desacoplada de estilo y lógica) y un diseño modular orientado a la inyección asíncrona de datos. A continuación, se detallan los principios técnicos y estructurales aplicados en las vistas principal y departamental:

3.1. Encabezado y barra de navegación

El encabezado del sitio se colocó dentro de un elemento header, que envuelve directamente el nav con la barra de navegación (clase navbar-expand-lg de Bootstrap). Se optó por anidar el nav dentro del header porque en esta primera entrega la navegación es simple, únicamente el enlace de regreso a Inicio, pero conforme se agregue el menú de acceso a las distintas zonas del país (occidental, central, oriental) este bloque seguirá siendo semánticamente correcto, ya que agrupa de forma clara todo lo relacionado a la navegación del sitio, separado del contenido informativo.

3.2. Área de contenido principal

Todo el contenido central de la página, tanto el texto introductorio como el mapa interactivo, se agrupó dentro de un único elemento main. Esta decisión se da debido a que en un documento HTML5 debe existir solamente un main por página, y su función es identificar cuál es el contenido exclusivo de esa vista, dejando fuera elementos que se repiten en todo el sitio como el encabezado o el pie de página.

3.3. Bloques temáticos con id’s

Dentro del main se definieron dos elementos section: uno identificado con el id intro, que contiene el título, el subtítulo, los párrafos introductorios y el mapa; y otro con el id division, que funciona como separador visual y de contenido hacia el siguiente bloque de la página ("Descubrí cada departamento"). Se utilizó section en lugar de div porque cada uno de estos bloques representa una

unidad temática independiente dentro del flujo del documento, y no una simple agrupación con fines de estilo.

3.4. Mapa SVG

Un punto importante de justificar es el uso del mapa como SVG embebido directamente en el HTML, en lugar de una imagen estática (img). Cada uno de los catorce elementos path que conforman los departamentos cuenta con un atributo id (por ejemplo SVAH, SVSA, SVCH) y un atributo name con el nombre real del departamento (Ahuachapán, Santa Ana, Chalatenango, entre otros). Esta forma de trabajar el mapa aporta varias ventajas frente a una imagen tradicional:

- Permite enlazar eventos de JavaScript (mouseover, click) a cada departamento de forma individual, sin depender de mapas de imagen (map/area), que son más rígidos y menos flexibles para aplicar interactividad.
- Permite aplicar directamente sobre cada figura los efectos de **elevación tridimensional (3D)** y transición dinámica al interactuar con el cursor, utilizando propiedades CSS avanzadas como transition, filter (mediante sombras proyectadas compuestas drop-shadow que simulan profundidad volumétrica) y transform (mediante escalado y traslación vertical que despegan visualmente el trazado del plano) aplicadas directamente sobre el propio elemento <path>.
- Mantiene la información de cada departamento disponible dentro del propio documento (DOM), lo que favorece la semántica y el acceso a esos datos, algo que en una imagen rasterizada quedaría oculto.

Adicionalmente, dentro del SVG se incluyeron dos grupos auxiliares: uno con id points, que guarda coordenadas de referencia geográfica en latitud y longitud, y otro con id label_points, que contiene un círculo por cada departamento con su nombre en el atributo class y su posición mediante cx y cy. Estos puntos permitirán, en fases posteriores del proyecto, colocar etiquetas o ventanas de información con los datos de cada departamento exactamente sobre el lugar correcto del mapa, sin necesidad de calcular esas posiciones de forma manual desde JavaScript.

3.5. Nomenclatura de clases e identificadores

Las clases utilizadas, como por ejemplo: encabezado-navegador, navbar-marca, contenido-texto, contenido-mapa y cta-texto, se nombraron siguiendo una forma descriptiva en español, coherente con el resto del proyecto, que indica la función del bloque dentro de la maqueta y no su apariencia visual. Esto permite que el CSS pueda modificarse más adelante sin que los nombres pierdan sentido ni se vuelvan confusos para cualquier integrante del equipo que continúe el desarrollo.

3.6. Carga de fuentes tipográficas

Se declaró la conexión anticipada (preconnect) hacia los dominios de Google Fonts antes de solicitar las fuentes Forum y Kranky. Esto reduce la latencia de conexión al servidor de fuentes antes de que el navegador solicite el archivo CSS correspondiente, lo cual mejora el tiempo de carga percibido en un sitio que depende de tipografías externas para su identidad visual.

3.7. Pie de página

El pie de página, que contiene el aviso de derechos de autor, se marcó como footer porque su contenido corresponde a información de cierre del documento y no al contenido principal de la página.

4. FUNDAMENTACIÓN DEL DISEÑO Y ESTILOS CSS

El diseño visual del sitio se apoya en Bootstrap 5.3.3 para la estructura de cuadrícula (grid) y los componentes base, complementado con una hoja de estilos propia (stylesMapa.css) que ajusta el comportamiento del mapa interactivo, la tipografía y los detalles visuales particulares del proyecto. A continuación, se explica el criterio detrás de cada bloque de estilos personalizado.

4.1. Ajuste responsivo del SVG del mapa

```
.contenido-mapa svg{
    width: 100%;
    height: auto;
    max-width: 100%;
    display: block;
    /*este css hace que el svg del mapa se ajuste al bloque que se le dio en
el boostrapgrid*/
}
```

Al tratarse de un SVG con dimensiones fijas definidas en su viewBox (1000 x 547), era necesario indicarle al navegador que el elemento debía comportarse como un bloque fluido dentro de la columna de Bootstrap que lo contiene. Se estableció el ancho al 100% y la altura en auto para que el mapa escale de forma proporcional según el espacio disponible en la columna, evitando que se deforme o se desborde en pantallas pequeñas. La propiedad display: block, por su parte, elimina el espacio en blanco inferior que los navegadores suelen añadir a los elementos SVG e imagen cuando se tratan como elementos en línea (inline), asegurando que el mapa se ajuste de forma precisa al contenedor.

4.2. Línea divisoria con texto centrado

```
.division-linea{
```

```
    display: flex;
    align-items: center;
    text-align: center;
    margin: 2rem 0;
    /*el flex hace que la linea izq el texto del span y la linea derecha
    vayan en una sola fila continua*/
}

.division-linea::before,
.division-linea::after{
    content:"";
    flex: 1;
    border-bottom: 1px solid #000;
 /*elementos fantasma solo con borde para hacer las lineas*/
}

.division-linea span{
    padding: 0 1rem;
    font-weight: 500;
    white-space: nowrap;
    font-family: 'Forum', serif;
    /*el nowrap evita que si el contenedor span se reduce
    el texto que esta en el span no se corte*/
}
```

Para el separador de sección "Descubrí cada departamento" se utilizó Flexbox junto con los pseudoelementos before y after para generar dos líneas decorativas a los costados del texto, sin necesidad de agregar elementos hr adicionales en el HTML. Cada pseudoelemento se declara con contenido vacío y flex: 1, lo que hace que ambos ocupen el espacio restante disponible en el contenedor de forma equitativa, empujando el texto central hacia el medio del bloque. El borde inferior de un pixel simula visualmente la línea. Sobre el texto contenido en el span se aplicó white-space: nowrap para evitar que, en pantallas angostas, la frase se corte o pase a una segunda línea, lo que rompería la composición visual del divisor; además se le asignó la tipografía Forum para mantener consistencia con el resto del contenido textual del sitio.

4.3. Jerarquía tipográfica (encabezados y párrafos)

```
h1, h2 {
  font-family: 'Kranky', cursive;
}

.contenido-texto p {
  font-family: 'Forum', serif;
```

```
  font-size: 1.1rem;
  line-height: 1.6;
}
```

Se definieron dos familias tipográficas con roles distintos dentro de la identidad visual del sitio. La fuente Kranky, de trazo manuscrito, se reservó exclusivamente para los títulos (h1 y h2), ya que su estilo transmite un carácter artesanal y cultural acorde a la temática del proyecto, pero por su misma naturaleza decorativa no es adecuada para bloques largos de texto, donde afectaría la legibilidad. Para el contenido de lectura extensa, dentro de contenido-texto, se utilizó en cambio la fuente Forum, una tipografía serif de rasgos más sobrios, a un tamaño de 1.1rem y con una altura de línea (line-height) de 1.6, valor que favorece la lectura de párrafos largos al dar suficiente espacio vertical entre renglones.

4.4. Barra de navegación fija y personalizada

```
.encabezado-navegador{
     font-family: 'Kranky', cursive;
     position: sticky;
    top: 0;
    z-index: 1000;
    /*hace que la abrra superior siempre este arriba*/
}


.navbar a {
  color:white;
  text-decoration: none;
  font-size: 1.1rem;
  font-weight: 600;
  padding: 0.75rem 1.5rem;
  background-color: rgba(177, 117, 32, 0.94);
  border-radius: 4px;
}

.navbar {
  background-image: url('../img/navbar.png');
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
}
```

Al contenedor del encabezado se le aplicó position: sticky junto con top: 0, de modo que la barra de navegación permanezca visible en la parte superior de la ventana sin importar cuánto se desplace el usuario dentro de la página; esto resulta útil considerando que el sitio contendrá varias secciones y subpáginas por departamento, y se busca que el acceso a la navegación esté siempre disponible. Se le asignó además un z-index alto (1000) para garantizar que el encabezado se mantenga por encima del resto del contenido, incluyendo el propio mapa SVG, evitando que otros elementos lo tapen al hacer scroll.

Los enlaces del nav se estilizaron como botones sólidos: se eliminó el subrayado por defecto, se les dio un color de fondo en tono tierra (rgba(177, 117, 32, 0.94)) que hace referencia a la paleta cultural y artesanal del proyecto, y se añadieron bordes redondeados junto con relleno interno (padding) para que cada enlace tenga un área de clic más generosa y un aspecto de botón más definido que el de un simple texto de navegación.

A la barra navbar se le agregó una imagen de fondo (navbar.png) configurada con background-size: cover y background-position: center, de manera que la imagen cubra siempre todo el ancho de la barra sin distorsionarse ni repetirse (background-repeat: no-repeat), adaptándose de forma consistente al ancho de pantalla del dispositivo del usuario. El color semitransparente aplicado a los enlaces sobre este fondo permite que la imagen se distinga de fondo, sin perjudicar la legibilidad del texto de navegación.

4.5. Identificadores de departamento, submenús y contenedores dinámicos

```css
/*resalta el borde del cuadro q contiene el mapa*/
.badge-departamento {
    display: inline-block;
    background-color: #26525c;
    color: #ffffff;
    font-family: 'Kranky', cursive;
    font-size: 1.25rem;
    padding: 0.5rem 1.75rem;
    border-radius: 6px;
    letter-spacing: 1px;
}
```

La clase .badge-departamento funciona como un distintivo visual que resalta el nombre del departamento seleccionado dinámicamente. Al asignarle display: inline-block, la etiqueta ajusta su ancho únicamente al contenido de texto interno. Su fondo azul verdoso (#26525c) y texto blanco

brindan un alto contraste contrastando con la tipografía Kranky, mientras que letter-spacing: 1px mejora el espaciado tipográfico entre caracteres.

4.6. Submenú navegable de categorías.

```
/*enlaces del submenú*/
.lista-submenus {
    margin-bottom: 0;
}

/*cada enlace del submenu se ve como boton, sin subrayado*/
.lista-submenus a {
    display: block;
    color: black;
    font-family: 'Forum', serif;
    font-size: 1.15rem;
    font-weight: 600;
    text-decoration: none;
    text-align: center;
    padding: 0.4rem 1.25rem;
    border-radius: 6px;
    transition: background-color 0.2s ease, color 0.2 ease;
}

.lista-submenus a:hover {
    background-color: #b17520;
    color: #ffffff;
}
```

Los enlaces dentro de .lista-submenus se diseñaron para comportarse como botones interactivos que permiten alternar entre las distintas categorías de información. La propiedad display: block asegura que el área cliqueable o táctil abarque todo el botón. Además, se añadió una transición suave (transition: 0.2s ease) al pasar el cursor sobre los enlaces (:hover), cambiando el fondo al tono ocre (#b17520) y el texto a blanco para dar retroalimentación visual al usuario.

4.7. Cajas contenedoras estáticas y placeholders dinámicos.

```
/*cajas contenedoras de contenido por js*/
.contenedor-placeholder {
    background-color: #d9d9d9;
    min-height: 160px;
    border-radius: 4px;
```

```
    padding: 1.5rem;
    display: flex;
    align-items: center;
    justify-content: center;
}
```

La clase .contenedor-placeholder define el área donde JavaScript inyectará dinámicamente el contenido cuando el usuario interactúe con el mapa. Se estableció un alto mínimo (min-height: 160px) y una estructura centradora basada en Flexbox (align-items: center y justify-content: center) sobre un fondo neutro (#d9d9d9), garantizando que la maqueta conserve su espacio visual predeterminado antes de cargar los datos de cada departamento.

4.8. Colorización por zona geográfica departamental

```
/*poniendole colores dependiendo de la zona departamental*/
#SVAH, #SVSA, #SVSO{
    fill: rgba(3, 221, 40, 0.801);
}

#SVUS, #SVUN, #SVSM, #SVMO{
    fill: #209eb1b3;
}

#SVSS, #SVLI, #SVPA, #SVCU, #SVCA, #SVCH, #SVSV{
    fill: #ff7c01db;
}
```

Para la distinción territorial del mapa de El Salvador, se aplicó la propiedad CSS fill sobre las etiquetas <path> del SVG identificadas por sus ISO correspondientes. Esto permite clasificar visualmente el país en sus tres zonas geográficas mediante colores con grados de transparencia: verde para la Zona Occidental (#SVAH, #SVSA, #SVSO), azul cian para la Zona Oriental (#SVUS, #SVUN, #SVSM, #SVMO) y naranja para la Zona Central/Paracentral (#SVSS, #SVLI, #SVPA, #SVCU, #SVCA, #SVCH, #SVSV)

4.9. Personalización de Popovers informativos de Bootstrap.

```
.pequeño-popover {
    --bs-popover-max-width: 130px;

    font-size: 0.65rem;
```

```
    font-family: cursive;


    --bs-popover-body-padding-x: 0.1rem;
    --bs-popover-body-padding-y: 0.1rem;
    --bs-popover-header-padding-x: 0.1rem;
    --bs-popover-header-padding-y: 0.1rem;
    --bs-popover-border-color: rgba(147, 150, 89);
    --bs-popover-bg: #B2AC88;
    --bs-popover-header-bg: rgba(147, 150, 89);;
    --bs-popover-header-font-size: 0.75rem;
    --bs-popover-body-color: #ffffff;
    --bs-popover-header-color: #ffffff;

    pointer-events: none;
}

.pequeño-popover .popover-header{
    font-family: 'Kranky', cursive;
}
```

Se sobrescribieron las variables CSS nativas del componente Popover de Bootstrap para adaptar los cuadros flotantes de información rápida que aparecen al desplazar el cursor sobre los departamentos. Se configuró un ancho máximo reducido (--bs-popover-max-width: 130px), una paleta cromática verde oliva (#B2AC88) y tipografía Kranky para el encabezado. La propiedad fundamental pointer-events: none previene que el emergente interfiera con la interacción del puntero sobre el elemento SVG subyacente.

4.10. Desplazamiento suave y desfases de secciones

```
/*para que el menu conoce mas aqui baje suave a cada seccion*/
html {
    scroll-behavior: smooth;
}

/*el navbar es sticky y tapaba el titulo de la seccion al bajar*/
.modulo-seccion {
    scroll-margin-top: 140px;
}
```

Al hacer clic en las opciones del menú de navegación, scroll-behavior: smooth activa una animación fluida de desplazamiento en la página hacia la sección correspondiente. Asimismo, la propiedad scroll-margin-top: 140px compensa la altura fija de la barra superior (sticky navbar), evitando que esta oculte el título o la parte superior del módulo destino al momento del anclaje.

4.11. Estructuración del contenedor de contenido dinámico

```
/*contenedor con el contenido real, reemplaza al placeholder gris*/
.contenedor-contenido {
    background-color: #d9d9d9;
    border-radius: 4px;
    padding: 1.5rem;
    font-family: 'Forum', serif;
    font-size: 1.1rem;
    line-height: 1.6;
}

.contenedor-contenido h3 {
    font-family: 'Kranky', cursive;
    margin-top: 1rem;
}

.contenedor-contenido h3:first-child {
    margin-top: 0;
}

#contenedor-texto-depto {
    text-align: center;
}
```

La clase .contenedor-contenido establece la presentación final del texto descriptivo inyectado desde JavaScript una vez que se selecciona un departamento. Define el estilo tipográfico (Forum), espaciados internos (padding: 1.5rem) y tratamiento especial para subtítulos h3 con la fuente Kranky. La regla :first-child elimina el margen superior innecesario en el primer título inyectado para lograr un alineamiento vertical idóneo dentro del contenedor.

4.12. Formateo y adaptabilidad visual de la silueta e imagenes

```
/*foto del departamento en la seccion de historia*/
.foto-depto{
    width: 100%;
```

```
    height: 260px;
    object-fit: cover;
}

/*el viewBox centra el dibujo dentro de esta caja*/
#contenedor-silueta-depto svg{
    height: 380px;
}

@media (max-width: 768px){
    #contenedor-silueta-depto svg{
        height: 260px;
    }
}
```

Las imágenes ilustrativas de cada departamento (.foto-depto) cuentan con object-fit: cover para recortarse y ajustarse proporcionalmente dentro de su marco sin sufrir distorsión. Para la silueta individualizada del departamento (#contenedor-silueta-depto svg), se especificó una altura base de 380px, regulada a 260px en pantallas móviles mediante una regla @media para dispositivos con ancho máximo de 768px.

4.13. Optimización para interacciones táctiles en dispositivos moviles

```
.contenido-mapa circle{
    pointer-events: none;
}
.contenido-mapa svg{
    touch-action: manipulation;
    -webkit-tap-highlight-color: transparent;
}

/*aviso que solo se ve en dispositivos sin hover (celulares y tablets)*/
.aviso-tactil{
    display: none;
    text-align: center;
    font-size: 0.9rem;
    margin-top: 0.5rem;
}

@media (hover: none){
    .aviso-tactil{
        display: block;
```

```
    }
}
```

Para perfeccionar la experiencia táctil en pantallas móviles y tabletas, se eliminó el resaltado por defecto al tocar el SVG mediante -webkit-tap-highlight-color: transparent y se configuró touch-action: manipulation para prevenir retardos o gestos no deseados (como doble toque para zoom). Mediante la consulta de medios @media (hover: none), el sitio detecta automáticamente si el dispositivo carece de puntero físico (mouse) para hacer visible únicamente en pantallas táctiles un aviso indicativo (.aviso-tactil).

4.14. Enmarcado responsivo para videos y créditos

```
/*se limita el ancho para que el video no ocupe toda la pantalla*/
.video-contenedor {
    max-width: 900px;
    margin: 0 auto;
    border-radius: 8px;
    overflow: hidden;
}

.creditos-video {
    text-align: center;
    font-family: 'Forum', serif;
    font-size: 0.9rem;
    margin-top: 0.5rem;
}
```

La clase .video-contenedor restringe el ancho máximo del reproductor multimedia a 900px para prevenir un tamaño excesivo en monitores de alta resolución, empleando margin: 0 auto para centrarlo horizontalmente. Sus esquinas redondeadas (border-radius: 8px) combinadas con overflow: hidden aseguran que el reproductor incrustado respete la curvatura del marco visual. El texto explicativo inferior (.creditos-video) se complementa en fuente Forum con alineación centrada

4.15. Efecto de elevación 3D en los departamentos del mapa

```
/*por default aplicamos estos presets para que se vea mas suave la
transicion del 3d*/
#features path {
    transition: transform 0.3s ease, filter 0.3s ease, fill 0.3s ease;
```

```
    transform-origin: center;
    transform-box: fill-box;
}

/*esta clase agg a cada path de deaprtamento cuando se detecte que se ha
seleccionado*/
.resaltado-3d{
filter: drop-shadow(0 6px 5px rgba(0,0,0,0.45));
transform: scale(1.04) translateY(-3px);
}
```

Para darle más interacción al mapa se creó un efecto de elevación que se activa cuando el usuario pasa el mouse sobre un departamento. La regla #features path prepara todos los departamentos con una transición de 0.3 segundos para los cambios de posición, sombra y color de relleno, y define que cualquier transformación se aplique desde el centro de la propia forma (transform-origin: center junto con transform-box: fill-box), no desde la esquina del SVG. Cuando JavaScript detecta el hover, le agrega la clase .resaltado-3d al departamento seleccionado. Esa clase lo agranda un 4 % (scale(1.04)), lo sube 3 píxeles (translateY(-3px)) y le pone una sombra suave debajo (drop-shadow), de modo que parece levantarse sobre el mapa. Al quitar el mouse, JavaScript elimina la clase y el departamento vuelve a su forma original con la misma transición suave.

5. Funcionamiento del sitio web y estructura de los archivos js.

5.1. menuPrincipal.js (Controlador del Mapa Interactivo y Navegación)

Gestiona la interacción del usuario en la pantalla principal (indexMapa.html). Asocia escuchadores de eventos a cada departamento del SVG para colorear la zona al pasar el puntero, maneja la inicialización y activación manual de los tooltips/popovers de Bootstrap, e implementa **compatibilidad con dispositivos móviles**: detecta si la interacción proviene de un mouse o de una pantalla táctil para que el primer toque muestre la etiqueta flotante y el segundo toque redirija a la página de detalle enviando el ID en la URL.

5.1.1. Fragmentos claves:

5.1.2. Lógica adaptable para pantallas táctiles y mouse.

```
//en celular no existe el hover, entonces el primer toque muestra el nombre
y el segundo entra a la pagina
let departamentoActivo = null; //guarda el departamento que ya se toco una
vez
```

```
let tipoPuntero = "mouse"; //se actualiza en pointerdown para saber si fue
mouse o dedo

departamento.addEventListener("pointerdown",(event)=>{
      tipoPuntero = event.pointerType;
   });

   //el hover solo se aplica con mouse, con el dedo no existe
   departamento.addEventListener("pointerenter",(event)=>{
      if (event.pointerType === "mouse") {
         resaltar(departamento);
      }
   });

   departamento.addEventListener("pointerleave",(event)=>{
      if (event.pointerType === "mouse") {
         quitarResaltado(departamento);
      }
   });

   //evento click que redirige a la pagina con la informacion
   departamento.addEventListener("click", () => {
      //con el dedo, si es el primer toque solo mostramos la informacion
      if (tipoPuntero !== "mouse" && departamentoActivo !== departamento) {
         if (departamentoActivo) {
            quitarResaltado(departamentoActivo);
         }
         departamentoActivo = departamento;
         resaltar(departamento);
         return;
      }
      window.location.href = `InfoDpto.html?depto=${id}`;
   });
});
//si tocan fuera de un departamento se quita la seleccion
document.addEventListener("click", (event)=>{
   if (departamentoActivo && !ArrayDepartamentos.includes(event.target)) {
      quitarResaltado(departamentoActivo);
      departamentoActivo = null;
   }
});
```

Este bloque implementa una lógica de interacción híbrida que adapta el mapa SVG según el dispositivo de entrada: mediante el evento pointerdown detecta si el usuario interactúa con ratón o

pantalla táctil (pointerType), permitiendo que en computadoras los efectos de resaltado y popovers funcionen al pasar el cursor (hover) con redirección inmediata al hacer clic, mientras que en dispositivos móviles compensa la ausencia de cursor con un mecanismo de doble toque donde la primera pulsación únicamente resalta la zona y almacena la referencia en departamentoActivo interrumpiendo la navegación, la segunda pulsación sobre el mismo elemento ejecuta la redirección hacia InfoDpto.html, y cualquier pulsación fuera del mapa restablece la selección y apaga los estilos activos.

5.2. Api.js (Capa de Servicios y Acceso a Datos)

Es el módulo encargado de toda la comunicación asíncrona y obtención de datos del proyecto. Implementa peticiones HTTP mediante XMLHttpRequest tradicional envueltas en Promise para permitir el uso moderno de async/await. Incluye un **mecanismo de respaldo (*fallback*)** (si el servidor en la nube de MockAPI falla o no hay conexión a internet, recurre automáticamente al archivo JSON local data/departamentos.json), un sistema de **caché en memoria** para evitar llamadas redundantes de red, y expone sus métodos al ámbito global mediante window.API.

5.2.1. Fragmentos Claves:

5.2.2. Petición AJAX asíncrona desacoplada con Promesas:

```
solicitarDatosAJAX(url) {
        return new Promise((resolve, reject) => {
            //Instancia del objeto XMLHttpRequest
            const xhr = new XMLHttpRequest();

            //Configuración de la petición HTTP GET asíncrona
            xhr.open("GET", url, true);

            //Manejador de evento al completarse la respuesta del servidor
            xhr.onload = function () {
                if (xhr.status === 200) {
                    try {
                        //Conversión de responseText a objeto JS mediante
JSON.parse()
                        const datos = JSON.parse(xhr.responseText);
                        resolve(datos);
                    } catch (errorParse) {
                        reject("Error al procesar el formato JSON: " +
errorParse);
                    }
                } else {
```

```
                    reject("Error en la petición. Código de estado HTTP: " +
xhr.status);
                }
            };

            //Manejo de error de conexión a nivel de red
            xhr.onerror = function () {
                reject("Error de red al intentar conectar con el
servidor.");
            };

            //Envío de la petición HTTP
            xhr.send(null);
        });
    },
```

Esta función encapsula una petición HTTP GET asíncrona mediante el objeto nativo XMLHttpRequest dentro de una promesa (Promise), lo que permite modernizar la arquitectura del proyecto para consumir datos con la sintaxis de async/await: al enviarse la solicitud, el manejador onload comprueba que el servidor responda con un código de estado exitoso (HTTP 200) para procesar el cuerpo de la respuesta con JSON.parse() y entregar los datos resueltos (resolve), mientras que canaliza y rechaza la promesa (reject) ante fallas en la sintaxis del JSON, códigos de error devueltos por el servidor o fallos de conexión capturados por el evento onerror.

5.2.3. Caché en memoria y fallback a JSON local:

```
async obtenerDepartamentos() {
        if (cacheDepartamentos && cacheDepartamentos.length > 0) {
            return cacheDepartamentos;
        }

        try {
            console.log("Realizando petición AJAX con XMLHttpRequest a:",
API_URL);
            cacheDepartamentos = await this.solicitarDatosAJAX(API_URL);
            return cacheDepartamentos;
        } catch (error) {
            console.warn("Fallo en la API REST externa. Intentando respaldo
local...", error);
            try {
                cacheDepartamentos = await
this.solicitarDatosAJAX(LOCAL_JSON_URL);
                return cacheDepartamentos;
```

```
            } catch (errorLocal) {
                console.error("No se pudo cargar la información:",
errorLocal);
                return [];
            }
        }
    },
```

Este método gestiona la recuperación eficiente y tolerante a fallos de la información departamental implementando una estrategia de almacenamiento en caché en memoria junto a un mecanismo de contingencia (*fallback*): primero comprueba si cacheDepartamentos ya posee datos para retornarlos de inmediato y evitar peticiones de red redundantes; si está vacía, intenta obtener los registros desde la API externa (API_URL) mediante solicitarDatosAJAX, y en caso de experimentar un fallo de conexión o caída del servidor remoto, captura la excepción para consumir automáticamente la copia de respaldo local (LOCAL_JSON_URL), devolviendo un arreglo vacío si ambas vías fallan para prevenir interrupciones críticas en la aplicación.

5.3. detalleDepto.js (Controlador de la Vista Departamental)

Es el script cerebro del sitio web InfoDpto.html. Se encarga de:

1. Leer y extraer el parámetro depto desde la URL del navegador (?depto=...).
2. Solicitar los datos a API.obtenerDepartamentoPorId(idDepto).
3. Dibujar la silueta vectorial consultando siluetas.js y reajustando la caja delimitadora (getBBox y viewBox) para que el departamento se encuadre y escale a la perfección.
4. Renderizar e inyectar dinámicamente toda la información en el DOM (reemplazando los contenedores de carga *placeholder* por historia, listas de municipios, tradiciones, gastronomía, lugares turísticos y fotografía representativa).

5.3.1. Fragmentos claves:

5.3.2. Lectura de parámetros URL y orquestación inicial:

```
//el id llega por la url, por ejemplo InfoDpto.html?depto=SVSS
const params = new URLSearchParams(window.location.search);
const idDepto = params.get("depto");

//arma una lista con los elementos de un arreglo
function crearLista(arreglo) {
```

```
    const items = arreglo.map(elemento => `<li>${elemento}</li>`).join("");
    return `<ul>${items}</ul>`;
}
async function iniciar() {
    //si entran a InfoDpto.html sin elegir departamento
    if (!idDepto) {
        mostrarError("No se eligió ningún departamento, regresa al mapa y
selecciona uno.");
        return;
    }

    const depto = await API.obtenerDepartamentoPorId(idDepto);

    if (!depto) {
        mostrarError(`No se encontró información para el departamento
${idDepto}.`);
        return;
    }

    mostrarDepartamento(depto);
}

iniciar();
```

Este bloque se encarga del arranque y control inicial de la página de detalle (InfoDpto.html), extrayendo primero el identificador del departamento desde los parámetros de búsqueda de la URL mediante URLSearchParams y definiendo una función auxiliar (crearLista) para transformar arreglos de datos en listas HTML no ordenadas (
...); a partir de ello, la función asíncrona iniciar orquesta el flujo de carga validando la existencia de dicho parámetro (desplegando un mensaje de advertencia con mostrarError si se ingresó directamente sin seleccionar una zona), solicita los datos correspondientes mediante API.obtenerDepartamentoPorId(idDepto) y, tras comprobar que el registro exista, delega el despliegue visual de la información a mostrarDepartamento(depto) para poblar dinámicamente toda la interfaz.

5.3.3. Renderizado vectorial y autoencuadre geométrico (viewBox):

```
function cargarSilueta(id) {
    const trazo = siluetas[id];

    if (!trazo) {
        console.warn("No hay silueta para " + id);
        return;
```

```
    }

    //el path conserva el id, asi el css le pone el color de su zona (#SVxx)
    contSilueta.innerHTML = `
        <svg xmlns="http://www.w3.org/2000/svg" stroke="#ffffff" stroke-
width="0.5" stroke-linejoin="round">
            <path id="${id}" d="${trazo}"></path>
        </svg>`;

    //ajustamos el viewBox al tamaño del departamento para que se vea
completo
    const svg = contSilueta.querySelector("svg");
    const caja = svg.querySelector("path").getBBox();
    svg.setAttribute("viewBox", `${caja.x} ${caja.y} ${caja.width}
${caja.height}`);
}
```

Esta función se encarga de renderizar de manera aislada y autoescalable la forma vectorial del departamento en la interfaz detallada: busca el trazado correspondiente en el diccionario siluetas mediante el identificador recibido, valida su existencia e inyecta dinámicamente un elemento <svg> en el contenedor conservando el atributo id en la etiqueta <path> para que las reglas CSS mantengan el color asignado a su zona geográfica, y finalmente invoca el método nativo .getBBox() para obtener las dimensiones y coordenadas exactas de la caja delimitadora del dibujo, asignándolas de inmediato al atributo viewBox para que la silueta se centre y escale a la perfección dentro del contenedor sin recortarse ni requerir ajustes manuales.

5.3.4. Inyección dinámica del contenido y eliminación de los placeholders de carga:

```
function llenarContenedor(contenedor, html) {
    contenedor.classList.remove("contenedor-placeholder");
    contenedor.classList.add("contenedor-contenido");
    contenedor.innerHTML = html;
}

function mostrarDepartamento(depto) {
    document.title = `${depto.nombre} - Nuestras Raíces`;
    badge.textContent = depto.nombre;

    contTexto.innerHTML = `
```

```
        <p>${depto.nombre} es un departamento ubicado en la zona
${depto.zona} de El Salvador, su cabecera departamental es
${depto.cabecera}.</p>
        <p>Tiene una población aproximada de
${depto.poblacion.toLocaleString("es-SV")} habitantes y una extensión de
${depto.extensionKm2.toLocaleString("es-SV")} km².</p>
    `;

    llenarContenedor(contHistoria, `
        <div class="row">
            <div class="col-md-8">
                <p>${depto.historia}</p>
                <h3 class="h5">Municipios</h3>
                ${crearLista(depto.municipios)}
            </div>
            <div class="col-md-4">
                <img src="img/${depto.id}.jpg" class="img-fluid rounded
foto-depto" alt="Imagen de ${depto.nombre}">
            </div>
        </div>
    `);

    llenarContenedor(contTradiciones, `
        <h3 class="h5">Fiestas patronales</h3>
        <p>${depto.fiestasPatronales}</p>
        <h3 class="h5">Personajes célebres</h3>
        ${crearLista(depto.personajesCelebres)}
    `);

    //el json todavia no tiene gastronomia, asi que por ahora puede venir
vacio
    if (depto.gastronomia && depto.gastronomia.length > 0) {
        llenarContenedor(contGastronomia, `
            ${crearLista(depto.gastronomia)}
        `);
    } else {
        llenarContenedor(contGastronomia, `<p>La información de gastronomía
de ${depto.nombre} se agregará pronto.</p>`);
    }

    llenarContenedor(contLugares, `
        <h3 class="h5">Lugares turísticos</h3>
        ${crearLista(depto.lugaresTuristicos)}
        <h3 class="h5">Ríos</h3>
        ${crearLista(depto.geografia.rios)}
```

```
        <h3 class="h5">Lagos y lagunas</h3>
        ${crearLista(depto.geografia.lagos)}
        <h3 class="h5">Volcanes y cerros</h3>
        ${crearLista(depto.geografia.volcanesCerros)}
    `);

    cargarSilueta(depto.id);
}
```

Este bloque se encarga de la presentación dinámica y actualización visual de la interfaz al recibir los datos de un departamento: mediante la función auxiliar llenarContenedor sustituye el estado de carga (contenedor-placeholder por contenedor-contenido) e inyecta el contenido HTML en cada módulo, mientras que mostrarDepartamento actualiza el título de la pestaña y la etiqueta principal, formatea las cifras de población y extensión territorial bajo el estándar regional salvadoreño (toLocaleString("es-SV")), e inyecta de forma estructurada en el DOM la reseña histórica junto a la fotografía representativa (img/${depto.id}.jpg), las tradiciones cívico-religiosas, la gastronomía típica mediante renderizado condicional, el catálogo geográfico-turístico y, por último, delega el dibujo vectorial de la zona invocando a cargarSilueta(depto.id).

5.4. siluetas.js (Repositorio de Geometrías Vectoriales)

Actúa como una base de datos estática de vectores gráficos. Contiene un objeto asociativo (siluetas) cuyas claves son los identificadores de los 14 departamentos (SVAH, SVSA, SVSS, etc.) y sus valores son las cadenas de comandos de trazado SVG (path d="..."). Su propósito es permitir el dibujado instantáneo e independiente de la silueta de cualquier departamento en la vista detallada sin depender de peticiones HTTP adicionales hacia archivos externos, garantizando funcionamiento incluso bajo el protocolo local file://.

6. BIBLIOGRAFÍA

Fundación Wikimedia. (2024). *Departamentos de El Salvador*. Obtenido de Wikipedia, la enciclopedia libre: https://es.wikipedia.org/wiki/Departamentos_de_El_Salvador

Ministerio de Turismo de El Salvador (MITUR). (2024). *Guía de Destinos Turísticos y Rutas Nacionales*. Obtenido de El Salvador Travel (CORSATUR): https://elsalvador.travel/

Ward, L. (2019). *Creating Interactive Maps in WordPress with MapSVG*. Obtenido de CSS-TRICKS: https://css-tricks.com/creating-interactive-maps-in-wordpress-with-mapsvg/

HTML: Lenguaje de Marcado de Hipertexto | MDN. (10 de junio de 2026). Obtenido de Mozilla.org https://developer.mozilla.org/es/docs/Web/HTML

SVG - Gráficos vectoriales escalables | MDN. (20 de junio de 2025). Obtenido de Mozilla.org: https://developer.mozilla.org/es/docs/Web/SVG

Conceptos Básicos de flexbox - CSS | MDN. (31 de marzo del 2026). Obtenido de Mozilla.org: https://developer.mozilla.org/es/docs/Web/CSS/Guides/Flexible_box_layout/Basic_concepts

Mark Otto, Jacob Thornton, and Bootstrap contributors. (2026). Get started with Bootstrap. Obtenido de getbootstrap: https://getbootstrap.com/docs/5.3/getting-started/introduction/

Comienza a usar la API de Google Fonts. (2025). Obtenido de Google for Developers: https://developers.google.com/fonts/docs/getting_started?hl=es-419

Mark Otto, Jacob Thornton, and Bootstrap contributors. (2026). Breakpoints. Obtenido de getbootstrap: https://getbootstrap.com/docs/5.3/layout/breakpoints/

Diseño receptivo - Aprende desarrollo web | MDN. (29 de abril del 2026). Obtenido de Mozilla.org: https://developer.mozilla.org/es/docs/Learn_web_development/Core/CSS_layout/Responsive_Design

Portal con los indicadores demográficos oficiales, tabulados de proyección de habitantes por departamento y censos nacionales.Obtenido de Oficina de Estadísticas y Censos del BCR https://estadisticas.bcr.gob.sv/

Sección que filtra sitios turísticos, atractivos naturales y patrimonio cultural categorizado por cada uno de los 14 departamentos. Obtenido por El Salvador Travel https://elsalvador.travel/esp/en/experiencias-tours-sv/

Sección institucional donde se publican las monografías departamentales y la división política administrativa oficial de la República. Obtenido por Instituto Geográfico y del Catastro Nacional (IGCN / CNR) https://www.cnr.gob.sv/instituto-geografico-y-del-catastro-nacional/

7. ANEXOS

Anexo 1. Datos generales de los departamentos de la Republica de El Salvador

Zona	Departamento	Cabecera	Población aproximada	Área (km^2)
Occidental	Ahuachapán	Ahuachapán	360,000 habitantes	1240
Occidental	Santa Ana	Santa Ana	590,000 habitantes	2023
Occidental	Sonsonate	Sonsonate	470,000 habitantes	1226
Central	La Libertad	Santa Tecla	750,000 habitantes	1653
Central	Chalatenango	Chalatenango	205,000 habitantes	2017
Central	Cuscatlán	Cojutepeque	250,000 habitantes	756.10
Central	San Salvador	San Salvador	1,750,000 habitantes	886.10
Central	La Paz	Zacatecoluca	330,000 habitantes	1224
Central	Cabañas	Sensuntepeque	165,000 habitantes	1104
Central	San Vicente	San Vicente	185,000 habitantes	1184
Occidental	Usulután	Usulután	370,000 habitantes	2130
Occidental	San Miguel	San Miguel	490,000 habitantes	2077
Occidental	Morazán	San Francisco Gotera	200,000 habitantes	1447
Occidental	La Union	La Union	265,000 habitantes	2074

Anexo 2. Enlace a repositorio de Github.

Repositorio de Github

Anexo 3. Enlace a página web proporcionado por Github.

